

AlienX 3D: A General $SO(3)$ -Equivariant Neural Operator for 3D Physical Fields

Author: Eric Yaka (Elbàlor / The Digital Necromancer)

Date: September 2026

Status: Architecture validated. 2D and 3D implementations complete. Physics-informed training, multi-regime generalization, and full rollout diagnostics documented. This paper defines AlienX 3D as a general geometric operator for 3D PDEs, not as a Navier-Stokes-specific model.

Abstract

AlienX 3D is a 421,507-parameter neural operator that learns the evolution of continuous 3D physical fields across multiple physical regimes without retraining. The architecture is not specific to any single PDE. It is built from four mechanisms that are each PDE-agnostic: (1) an $SO(3)$ -equivariant local frame derived from the gradient and Hessian of a chosen scalar field, (2) Quantum Self-Attention (QSA), a complex-valued attention mechanism operating in CP^{n-1} with a geometric phase, (3) per-sample non-dimensionalization with a regime conditioning channel, and (4) a normalized conservation penalty that enforces a physical invariant in Fourier space. The specific fields, regime parameter, and invariant are supplied by the user as an outer shell. Everything else — the frame builder, the QSA blocks, the equivariant output, the training protocol — is shared across problems.

Demonstrated on incompressible 3D Navier-Stokes, trained on the Taylor-Green Vortex at three Reynolds numbers (300, 628, 1200) with a single model. On single-step prediction, AlienX 3D achieves 0.46% relative RMSE across all three regimes (0.16% at $Re=300$). A QSA ablation degrades validation MSE by $32.81\times$, confirming that the complex attention stream is the load-bearing mechanism. Autoregressive rollout is stable across the full 73-step trajectory with bounded energy drift (0.56% / 2.68% / 5.81% at $Re=300/628/1200$). The operator is grid-agnostic: the same weights evaluate at $G=32$ and $G=64$ without retraining. Divergence-free is preserved by construction throughout the rollout (Biot-Savart projection; float32 round-off $\sim 2.7e-6$). The operator also generalizes zero-shot to a held-out Reynolds number ($Re=900$): 0.53% single-step relative RMSE, rollout invariants on the interpolation

trend between the trained regimes, and a $4.74\times$ degradation when the complex phase channel is ablated — the model interpolates the physics of regimes it never saw. The architecture is a substrate for 3D operator learning — NS was the hardest test in the class.

1. Introduction

1.1 The state of 3D operator learning

Neural operators promise to replace expensive numerical solvers with learned surrogates. FNO, DeepONet, and graph neural operators have demonstrated this convincingly on 2D problems. In 3D, the field is sparse:

most published neural operators operate in 2D at 64^2 – 256^2 with 0.3M–67M parameters. The 3D results that exist use 5M–50M parameters and rarely generalize across physical regimes.

The gap is not computational. It is architectural. Most operators learn a fixed-grid approximation. They overfit to a single resolution, a single viscosity, a single flow family. Change any of these and the operator must be retrained.

AlienX 3D closes this gap by making the geometry structural rather than learned. Rotation, scale, and translation are not features to memorize. They are coordinate artifacts that vanish before learning begins.

1.2 The core insight

The network should not learn geometry from data. It should be born with geometry.

Every feature that reaches a learnable weight in AlienX 3D has been made rotation- and scale-invariant by construction. The weights never see absolute rotation or absolute scale — only frame-relative phase and locally-normalized distance. Rotation, scale, and translation are not features. They are coordinate artifacts, removed before the first linear layer.

1.3 What AlienX 3D is not

AlienX 3D is not a Navier-Stokes solver. It is a 3D operator, demonstrated on NS. The NS-specific parts of this paper are:

- The choice of scalar field: $k = |\omega|$, vorticity magnitude.
- The vector fields fed to QSA: $(\nabla k, \omega, u)$.

- The regime parameter: Re .
- The invariant: $\nabla \cdot \mathbf{u} = 0$.

Swap those four and the same architecture applies to MHD, Rayleigh-Bénard, compressible NS, Gross-Pitaevskii, elasticity, and combustion. Section 10 spells out the substitutions.

1.4 Contributions

1. A general $SO(3)$ -equivariant 3D neural operator architecture.
2. Quantum Self-Attention with geometric phase derived from local frame coordinates (first application to a 3D PDE).
3. Multi-regime generalization: three Reynolds numbers in a single 421K-parameter model with no retraining. Zero-shot transfer to a held-out Reynolds number ($Re=900$, 0.53% single-step) shows the model interpolates the regime family rather than memorizing trajectories.
4. A $32.81\times$ QSA ablation ratio, confirming the complex attention stream as the load-bearing mechanism, and a $4.74\times$ phase-only ($g_i=0$) ablation on held-out $Re=900$ isolating the complex phase mechanism specifically.
5. Rollout stability across a 73-step trajectory — $24\times$ the training horizon — with energy bounded under 6% at all Re .
6. Grid-agnosticism: same weights, same rule, different grids.
8. Zero-shot generalization to a held-out Reynolds number ($Re=900$), with rollout invariants landing on the interpolation trend (§7.6).
7. Complete debugging log of every numerical issue encountered and fixed (§8).

2. Prerequisites: What is an operator?

An operator is a map that takes a function as input and returns a function as output. The Navier-Stokes equations themselves are an operator: given a velocity field at time t , they produce the velocity field at time $t + \Delta t$.

A neural operator is a neural network that learns to approximate this map. It is trained on many pairs $(\mathbf{u}(t), \mathbf{u}(t + \Delta t))$ and learns to predict the second from the first. Once trained, it can evaluate on new initial conditions far faster than the numerical solver that generated the training data.

The challenge is that a velocity field is not a fixed-size vector. It is a function on a continuous domain. Two fields at different resolutions represent the same physical object. A naive network would need to be retrained for each resolution.

AlienX 3D solves this by operating on continuous manifolds instead of fixed grids. Every node is a point in $\mathbb{R}^3$. Every edge is a physical displacement. The learnable weights operate on frame-relative features that are resolution-independent.

3. Mathematical Foundations

3.1 The operator learning setup

Let $u(x, t): \Omega \times \mathbb{R} \rightarrow \mathbb{R}^d$ be a physical field on a periodic domain $\Omega \subset \mathbb{R}^3$. The governing PDE is:

$$\partial u / \partial t = F(u, \nabla u, \nabla^2 u, \dots; \theta_{\text{phys}})$$

where θ_{phys} is a set of physical parameters (viscosity, conductivity, etc.). We want to learn a parameterized operator G_ϕ that maps the state at time t to the state at time $t + \Delta t$:

$$G_\phi : u(\cdot, t) \rightarrow u(\cdot, t + \Delta t)$$

AlienX 3D consumes $(k, \nabla k, H(k), v_i)$ and produces the next-step state, where:

- k is a scalar field of physical meaning (magnitude of a natural vector field).
- v_i are the state variables and derived quantities the operator must evolve.

This formulation is PDE-agnostic: the specific fields are user choices.

3.2 The SO(3)-equivariant local frame

At each node of the computational grid, we construct an orthonormal frame $F = [e_1 \mid e_2 \mid n] \in \text{SO}(3)$ from the local scalar field k and its derivatives. The construction has six stages.

Stage 1. Primary anchor from the gradient. Let ∇k be the spatial gradient. Define:

$$n_{\text{grad}} = \nabla k / (|\nabla k| + \epsilon)$$

This is the natural normal at each node. It points along the direction of steepest ascent of k .

Stage 2. Fallback anchor from spatial inertia. When $|\nabla k|$ is small (flat regions where the gradient is ill-defined), fall back to the leading eigenvector of the inertia tensor of the field:

$$S = \sum_i w_i x_i x_i^T, \quad w_i = \text{softmax}(k_i / \tau)$$

where τ is a small temperature and x_i are node coordinates. The leading eigenvector is the direction of maximum spatial extent of the field — a property of the field itself, not of the grid. Degenerate eigenvalue spectra are handled with an adaptive jitter ladder (retry with $S + j \cdot I$ for j in $[0, 1e-4, 1e-3, 1e-2, 5e-2]$).

Critical detail: the perturbation matrix must be isotropic. An anisotropic perturbation (e.g. $\text{diag}([1e-3, 2e-3, 3e-3])$) breaks $SO(3)$ equivariance because $R(\epsilon I)R^T = \epsilon I$ only holds for isotropic ϵI . We use $\text{pert_mat} = 1e-3 \cdot I$.

Stage 3. Smooth gate between anchors. Blend the two anchors:

$$g = \sigma(|\nabla k| - \epsilon_{\text{grad}}) / \tau_{\text{gate}} \\ n = \text{normalize}(g \cdot n_{\text{grad}} + (1 - g) \cdot n_{\text{inertia}})$$

The gate is differentiable, so gradients flow through both branches.

Stage 4. In-plane orientation from the Hessian. The gradient pins one axis (n). A second intrinsic direction is needed to complete the frame. Build two orthonormal tangent vectors u_1, u_2 perpendicular to n . Project the Hessian onto the tangent plane:

$$a = u_1^T H u_1, \quad b = u_1^T H u_2, \quad c = u_2^T H u_2$$

The principal eigenvector of this 2×2 tangent Hessian lies at angle:

$$\theta = \frac{1}{2} \text{atan2}(2b, a - c) \\ e_1 = \cos(\theta) u_1 + \sin(\theta) u_2$$

This principal direction is the direction of maximum curvature in the plane perpendicular to ∇k . It is basis-independent: choose any (u_1, u_2) , rotate them by φ , θ shifts by $-\varphi$, and e_1 comes out the same. This is what makes e_1 equivariant regardless of how u_1, u_2 are chosen.

Stage 5. Sign lock. The eigenvector is defined up to sign. Lock it:

$$Hn = H \cdot \nabla k \\ Hn_{\text{tan}} = Hn - (Hn \cdot n) n \\ \text{sgn} = \text{sign}(e_1 \cdot Hn_{\text{tan}}) \\ e_1 \leftarrow \text{sgn} \cdot e_1$$

Stage 6. Complete the frame. Set $e_2 = n \times e_1$. The frame $F = [e_1 \mid e_2 \mid n]$ is in $SO(3)$ by construction. A phase confidence is computed from the eigenvalue gap:

$$\delta = \sqrt{(a - c)^2 + 4b^2}$$

$$p_{\text{conf}} = \sigma((\delta - \epsilon_{\text{conf}}) / \tau_{\text{conf}})$$

This confidence controls how much phase information is preserved in attention. In near-degenerate regions (weak curvature anisotropy), phase is unreliable and is down-weighted.

Equivariance. Under a global rotation R :

- $\nabla k \rightarrow R \nabla k$
- $H \rightarrow R H R^T$
- Inertia $S \rightarrow R S R^T$
- Eigenvectors of S rotate by R
- $n \rightarrow R n, e_1 \rightarrow R e_1, e_2 \rightarrow R e_2$

The full frame transforms as $F \rightarrow R F$. Every downstream projection into F is therefore rotation-invariant. This is the equivariance property that makes AlienX 3D structurally correct.

3.3 Quantum Self-Attention (QSA)

Attention in AlienX 3D operates in complex projective space CP^{n-1} , not in the real probability simplex. This is a departure from standard transformer attention.

Complex projections. Given node hidden states $h_i \in \mathbb{R}^C$, project to complex Q, K, V :

$$Q_i = W_q h_i = W_{qr} h_{ir} + i \cdot W_{qi} h_{ir}$$

$$K_j = W_k h_j \text{ (similarly)}$$

$$V_j = W_v [h_j \parallel \nabla k_{\text{local}_j} \parallel \omega_{\text{local}_j}] \text{ (similarly)}$$

Hermitian inner product. The interference between query and key is:

$$S_{ij} = Q_i \cdot \text{conj}(K_j)$$

$$= (Q_r \cdot K_r + Q_i \cdot K_i) + i(Q_i \cdot K_r - Q_r \cdot K_i)$$

The real part is a standard dot product. The imaginary part is antisymmetric under swap of i and j . This asymmetric component encodes directional relationships that softmax attention cannot represent.

Geometric phase. Each neighbor j of node i has a relative displacement in the local frame:

```

r_ij = x_j - x_i
dx_local = r_ij · e1_i

dy_local = r_ij · e2_i
dz_local = r_ij · n_i

```

The relative displacement is computed from fixed stencil offsets, not from coordinate differences. This matters for periodic domains: coordinate differences wrap around the box boundary and produce near- 2π displacements where the true physical displacement is small. Using the stencil offsets (the same offsets used to build the neighbor list) gives exact small vectors everywhere. The in-plane angle and off-plane taper are:

```

θij = atan2(dylocal, dxlocal)
woff = exp(-dzlocal2 / σz2)

```

The complex geometric phase is:

```

gij = woff · (cos θij + i · sin θij · pconf,i) = grij + i · giij

```

This is not a learned wavevector. It is derived from the local frame coordinates. This eliminates the phase-drift instability that arises when phases are free parameters.

Phase modulation and ℓ^2 normalization. Modulate the interference:

```

s'ij = sij · gij

```

Then normalize by ℓ^2 over the key dimension:

```

αij = s'ij / √(Σj |s'ij|2 + ε)

```

This replaces softmax. It preserves phase, bounds the norm, and avoids the exponential overflow that softmax is vulnerable to at large grid sizes.

Coherent superposition. The output is a complex sum:

```

Hi = Σj αij · Vj

```

Constructive interference occurs when $\arg(\alpha_{ij}) + \arg(V_j) \approx 0$. Destructive interference occurs when the sum $\approx \pi$. This is native subtraction inside the attention mechanism — the network can cancel a bad idea rather than drown it out.

Residual. The block output is:

```

hi ← hi + 0.9 · LayerNorm(Hi)

```

The 0.9 residual scaling keeps the attention contribution slightly below the identity, stabilizing early training.

3.4 Per-sample non-dimensionalization

Physical fields have arbitrary amplitude. A velocity field with $U = 1$ and a field with $U = 1000$ are physically identical up to scale. To make the operator scale-invariant, normalize each sample by its own characteristic amplitude:

```
U =  $\sqrt{\langle |u|^2 \rangle}$       (RMS velocity, or analogue for other PDEs)
 $\tilde{\omega} = \omega / U$ 
 $\tilde{u} = u / U$ 
 $\tilde{k} = |\omega|$ 
```

After non-dimensionalization, the operator sees identical inputs regardless of the amplitude of the underlying physical field. The output is rescaled back by U at inference.

After non-dimensionalization, apply global standardization: each feature channel is centered and scaled to unit variance across the dataset.

3.5 Regime conditioning

The regime parameter (Reynolds number, Rayleigh number, Mach number, etc.) is injected as a scalar channel:

```
re_channel =  $\log(\text{Re} / \text{Re}_{\text{max}})$ 
```

This allows the network to distinguish regimes without magnitude bias. Without this channel, the model would conflate a high-Re flow with a low-Re flow that happens to have similar local structure.

The channel is broadcast to every node and concatenated with the other input features.

3.6 Normalized conservation penalty

Every 3D PDE of interest has a physical invariant: incompressibility for NS ($\nabla \cdot \mathbf{u} = 0$), continuity for compressible flow, Gauss's law for MHD ($\nabla \cdot \mathbf{B} = 0$). Enforce this invariant as a soft penalty in the loss.

The naive formulation $\text{mean}((\nabla \cdot \mathbf{u})^2)$ is dominated by high-frequency modes because the divergence operator multiplies by wavenumber k . Normalize:

```
L_div =  $\text{mean}(|\text{ifft}(\mathbf{i} \cdot \mathbf{K} \cdot \hat{\mathbf{u}} / |\mathbf{K}|)|^2)$ 
```

where K is the wavenumber vector and $\hat{u}$ is the Fourier transform of the predicted velocity. This normalizes the divergence operator to have MSE-like units, allowing the weight λ_{div} to be chosen independently of grid size.

3.7 Equivariant output

The operator produces a vector output v_{local} in the local frame. To produce a rotation-equivariant output in the global frame, rotate back through F :

$$v_{\text{global},i} = F_i \cdot v_{\text{local},i}$$

Under a global rotation R , $F_i \rightarrow R F_i$ and $v_{\text{local},i}$ is invariant (because the projections into F are invariant). Therefore $v_{\text{global},i} \rightarrow R v_{\text{global},i}$. The output is exactly rotation-equivariant.

4. Architecture

AlienX 3D consists of:

1. An input projection that lifts 12 scalar and vector channels to hidden dimension $H = 128$.
2. $L = 4$ stacked QSA-ISN blocks.
3. An output projection that maps $H \rightarrow 3$ (vorticity, in the local frame).
4. An equivariant back-rotation via the frame F .

Input channels per node (for the NS demonstration):

```
# Channel Shape
1  $\tilde{k} = \sqrt{|\omega|}$  [B, N]
2  $\sqrt{|\nabla \tilde{k}|}$  [B, N]
3  $\nabla \tilde{k}$  in local frame [B, N, 3]
4  $\tilde{\omega}$  in local frame [B, N, 3]
5  $\tilde{u}$  in local frame [B, N, 3]
6  $\log(\text{Re} / \text{Re}_{\text{max}})$  [B, N]
Total 12 channels
```

Each QSA-ISN block performs:

1. Complex Q, K, V projections.
2. Neighbor gather over 26 isotropic neighbors.
3. Hermitian inner product S_{ij} .
4. Phase modulation by geometric phase g_{ij} .

5. ℓ_2 normalization over keys.
6. Coherent complex superposition.
7. Output projection + LayerNorm + 0.9 residual.

The final hidden state is projected to a 3-vector in the local frame, then rotated back to global coordinates via F .

5. Training Protocol

5.1 K-unroll with Biot-Savart feedback

Naive single-step training produces a model that fails under autoregressive rollout. The model learns to predict $t+1$ from the true state at t , but at inference it must predict $t+1$ from its own prediction at t — an out-of-distribution input.

AlienX 3D is trained with K -step unroll. At each training step:

```
u_curr = true u(t0)
for step in 1..K:
  ω_pred = model(features(u_curr))
  u_pred = Biot-Savart(ω_pred)      ← exact spectral recovery
  loss += L_step(ω_pred, ω_true(t0+step))
  u_curr = u_pred
```

The model sees its own predictions at steps 2 through K during training. Gradients flow through the unroll. The Biot-Savart recovery step ($\omega \rightarrow u = \nabla \times \nabla^{-2} \omega$) is exact in spectral space, so the velocity field is divergence-free at machine precision at every unroll step.

5.2 K curriculum

Training begins at $K=1$ and ramps through $K=2$, $K=3$, and on to $K=5$ over 120 epochs:

Stage	Epochs	K
1	1–15	1
2	16–35	2
3	36–60	3
4	61–120	5

The warmup prevents early instability. Each K increment causes a temporary validation spike (expected — the model is being asked a harder question) followed by recovery below the previous minimum.

Important note on the results reported in §7. The K=5 run was performed on Google Colab free tier. The platform's idle timeout terminated the session at approximately epoch 55 of 120. The reported single-step RMSE, QSA ablation, and rollout diagnostics are all from the K=3 stage of this run — specifically the epoch-55 checkpoint at val RMSE 0.50%. The K=5 stage is specified as the recommended protocol and is the immediate next training phase. The full $K=1 \rightarrow K=2 \rightarrow K=3$ training trajectory that produced the reported results is documented in §7.4.

This does not weaken the results. The K=3 training already achieves single-step accuracy an order of magnitude beyond published 3D operators on their own benchmarks (FNO-3D reports 24% nRMSE on PDEBench — a different benchmark and flow family, so no cross-benchmark ratio is claimed), holds a 73-step autoregressive rollout at bounded energy drift, and generalizes zero-shot to a held-out Reynolds number (§7.6). The K=5 curriculum is expected to improve the $Re=1200$ rollout further, and is left as future work (§11).

5.3 Physics-informed loss

The full loss:

```
L = λ_MSE · MSE(ω_pred, ω_true)
+ λ_E · (log E_pred - log E_true)²
+ λ_Z · (log Z_pred - log Z_true)²
+ λ_div · L_div(u_pred)
+ λ_spec · L_spectral
```

with $\lambda_{MSE} = 1.0$, $\lambda_E = 2.0$, $\lambda_Z = 5.0$, $\lambda_{div} = 0.05$, $\lambda_{spec} = 0.3$.

Energy and enstrophy in log-space. Physical E and Z decay exponentially in TGV at late times. Log-space is the natural coordinate for exponential decay. Clamping at $1e-8$ prevents numerical blowup as $E \rightarrow 0$.

Why Z is weighted heavier than E: $dE/dt = -2\nu Z$. Enstrophy is the cause, energy is the effect. Matching Z produces correct E as a consequence.

Spectral loss. Matches $E(k)$ and $Z(k)$ shell-by-shell, not just the integrals. Computed on integer-mode bins — the previous version used angular wavenumbers (0..174 at $G=32$) with integer bins (0..16), which silently zeroed every bin except DC. The current version uses integer mode magnitude $k_{mag_modes} = \sqrt{K^2} / 2\pi$ in bins [0, 17].

Divergence DC fix. The projection operator uses $K2[0,0,0] = 1.0$ to avoid division by zero. This hack leaks into the enstrophy spectrum: the DC bin gets a spurious $K^2=1$ contribution. Fixed with explicit $Z_density[..., 0, 0, 0] = 0$ before binning.

6. Implementation

6.1 Spectral DNS solver

The NS data is generated by a pseudo-spectral solver with IMEX-Euler time integration:

```
def make_wavenumbers(G, device):
    kx = 2*math.pi*torch.fft.fftfreq(G, d=1.0/G, device=device)
    ky = 2*math.pi*torch.fft.fftfreq(G, d=1.0/G, device=device)
    kz = 2*math.pi*torch.fft.fftfreq(G, d=1.0/G, device=device)
    KX, KY, KZ = torch.meshgrid(kx, ky, kz, indexing='ij')

    K2 = KX**2 + KY**2 + KZ**2
    K2[0,0,0] = 1.0
    return KX, KY, KZ, K2

def projection(u_hat, v_hat, w_hat, KX, KY, KZ, K2):
    kd = KX*u_hat + KY*v_hat + KZ*w_hat
    return (u_hat - KX*kd/K2, v_hat - KY*kd/K2, w_hat - KZ*kd/K2)

def nonlinear_term(u_hat, v_hat, w_hat, KX, KY, KZ, K2):
    u = torch.fft.ifftn(u_hat, dim=(-3,-2,-1)).real
    v = torch.fft.ifftn(v_hat, dim=(-3,-2,-1)).real
    w = torch.fft.ifftn(w_hat, dim=(-3,-2,-1)).real
    du_dx = torch.fft.ifftn(1j*KX*u_hat, dim=(-3,-2,-1)).real
    du_dy = torch.fft.ifftn(1j*KY*u_hat, dim=(-3,-2,-1)).real
    du_dz = torch.fft.ifftn(1j*KZ*u_hat, dim=(-3,-2,-1)).real
    dv_dx = torch.fft.ifftn(1j*KX*v_hat, dim=(-3,-2,-1)).real
    dv_dy = torch.fft.ifftn(1j*KY*v_hat, dim=(-3,-2,-1)).real
    dv_dz = torch.fft.ifftn(1j*KZ*v_hat, dim=(-3,-2,-1)).real
    dw_dx = torch.fft.ifftn(1j*KX*w_hat, dim=(-3,-2,-1)).real
    dw_dy = torch.fft.ifftn(1j*KY*w_hat, dim=(-3,-2,-1)).real
    dw_dz = torch.fft.ifftn(1j*KZ*w_hat, dim=(-3,-2,-1)).real
    ax = u*du_dx + v*du_dy + w*du_dz
    ay = u*dv_dx + v*dv_dy + w*dv_dz
    az = u*dw_dx + v*dw_dy + w*dw_dz
    ax_hat = torch.fft.fftn(ax,
    dim=(-3,-2,-1))
    ay_hat = torch.fft.fftn(ay, dim=(-3,-2,-1))
    az_hat = torch.fft.fftn(az, dim=(-3,-2,-1))
```

```

G = u_hat.shape[-1]; kmax = G // 2
kmask = ((torch.abs(KX) <= 2*kmax/3) &
(torch.abs(KY) <= 2*kmax/3) &
(torch.abs(KZ) <= 2*kmax/3))
ax_hat, ay_hat, az_hat = ax_hat*kmask, ay_hat*kmask, az_hat*kmask
ax_hat, ay_hat, az_hat = projection(ax_hat, ay_hat, az_hat, KX, KY, KZ, K2)
return -ax_hat, -ay_hat, -az_hat

def tgv_imex_step(u_hat, v_hat, w_hat, KX, KY, KZ, K2, nu, dt):

nhx, nhx, nhz = nonlinear_term(u_hat, v_hat, w_hat, KX, KY, KZ, K2)
vf = 1.0 + dt*nu*K2
return (u_hat + dt*nhx)/vf, (v_hat + dt*nhx)/vf, (w_hat + dt*nhz)/vf

```

The solver uses Orszag 2/3 de-aliasing and Leray projection at every step, giving divergence-free fields at every snapshot.

6.2 FrameBuilder

```

class FrameBuilder(nn.Module):
def __init__(self):
super().__init__()
# Isotropic perturbation -  $R(\epsilon I) R^T = \epsilon I$  for any rotation  $R$ .

# Anisotropic perturbation would break  $SO(3)$  equivariance.
self.register_buffer('pert_mat', 1e-3 * torch.eye(3).unsqueeze(0))
self.tau = 1e-3
self.epsilon_grad = 1e-4

def forward(self, grad_k, H, coords, k):
with torch.no_grad():
B, N, _ = coords.shape
mag = grad_k.norm(dim=-1, keepdim=True)
n_grad = grad_k / (mag + 1e-8)

w = torch.softmax(k.unsqueeze(-1), dim=1)
S = torch.einsum('bnd,bnc->bdc', coords*w, coords)
S = S / (S.norm(dim=(-2,-1), keepdim=True) + 1e-8)

S = S + self.pert_mat
S = 0.5 * (S + S.transpose(-1, -2))

evec = None
for j in [0.0, 1e-4, 1e-3, 1e-2, 5e-2]:
try:
S_try = S if j == 0 else S + j*torch.eye(3, device=S.device).unsqueeze(0)
if j > 0:
S_try = 0.5 * (S_try + S_try.transpose(-1, -2))
_, ev = torch.linalg.eigh(S_try)

```

```

evec = ev
break
except RuntimeError:
continue
if evec is None:
evec = torch.eye(3, device=S.device).unsqueeze(0).expand(

B, -1, -1)

fb = evec[..., -1].unsqueeze(1).expand(-1, N, -1)

gate = torch.sigmoid((mag - self.epsilon_grad) / self.tau)
n = F.normalize(gate*n_grad + (1-gate)*fb, dim=-1)

# Tangent reference: inertia eigenvector least aligned with n
evec_exp = evec.unsqueeze(1).expand(-1, N, -1, -1)
dots = torch.einsum('bnij,bni->bnj', evec_exp, n)
abs_dots = dots.abs()
best_idx = abs_dots.argmax(dim=-1)
ref = evec_exp.gather(

3, best_idx.unsqueeze(-1).unsqueeze(-1).expand(-1, -1, 3, 1)
).squeeze(-1)

u1 = F.normalize(torch.cross(n, ref, dim=-1), dim=-1)
u2 = torch.cross(n, u1, dim=-1)

Hu1 = torch.einsum('bnij,bnj->bni', H, u1)
Hu2 = torch.einsum('bnij,bnj->bni', H, u2)
a = (u1*Hu1).sum(-1); b = (u1*Hu2).sum(-1); c = (u2*Hu2).sum(-1)
th = 0.5 * torch.atan2(2*b, a - c + 1e-8)
e1 = torch.cos(th).unsqueeze(-1)*u1 + torch.sin(th).unsqueeze(-1)*u2

Hn = torch.einsum('bnij,bnj->bni', H, grad_k)
Ht = Hn - (Hn*n).sum(-1, keepdim=True)*n
sgn = torch.sign((e1*Ht).sum(-1, keepdim=True))
sgn = torch.where(Ht.norm(dim=-1, keepdim=True) > 1e-6, sgn,
torch.ones_like(sgn))
e1 = F.normalize(e1*sgn, dim=-1)
e2 = torch.cross(n, e1, dim=-1)

disc = torch.sqrt((a-c)**2 + 4*b**2 + 1e-8)
conf = torch.sigmoid((disc - 1e-3) / 1e-4)
return e1, e2, n, conf

```

6.3 Geometric phase precompute

```

def precompute_geometry(coords, e1, e2, n, pc, knn_idx, sigma_z, G):
"""

```

Stencil offsets give exact small-displacement vectors.

No min-image wrap – the circular stencil padding handles periodicity correctly, and coordinate differences would produce near- 2π vectors at boundaries, breaking the in-plane angle computation.

```
"""
B, N, _ = coords.shape
K = knn_idx.shape[-1]
dilation = max(1, G // 16)

offsets = torch.tensor(ISO_OFFSETS_3D, device=coords.device, dtype=torch.float32)
offsets = offsets * dilation * (2*math.pi / G)
rel = offsets.view(1, 1, K, 3).expand(B, N, K, 3)

bidx = torch.arange(B, device=coords.device).view(B, 1, 1)
bx = bidx.expand(-1, N, K)
with torch.no_grad():
    dx1 = (rel * e1.unsqueeze(2)).sum(-1)
    dy1 = (rel * e2.unsqueeze(2)).sum(-1)
    dz1 = (rel * n.unsqueeze(2)).sum(-1)
    th = torch.atan2(dy1, dx1)
    wo = torch.exp(-(dz1**2) / (sigma_z**2))
    gr = wo * torch.cos(th)

gi = wo * torch.sin(th) * pc.unsqueeze(-1)
return gr, gi, bx
```

6.4 QSA block

```
class QSA_ISNBlock3D(nn.Module):
    def __init__(self, C, vec_dim=3, heads=4, eps=1e-8, chunk=0):
        super().__init__()
        self.C = C; self.H = heads; self.D = C//heads
        self.eps = eps; self.chunk = chunk
        self.q_proj = nn.Linear(C, C)
        self.k_proj = nn.Linear(C, C)
        self.v_proj = nn.Linear(C + 2*vec_dim, 2*C)
        self.out_proj = nn.Linear(2*C, C)

        self.norm = nn.LayerNorm(C)
        self.last_im_re_ratio = 0.0

    def _chunk_forward(self, h_c, q_c, k_node, v_node, gr_c, gi_c, bx_c, knn_c):
        B, cs, C = h_c.shape
        K_j = k_node[bx_c, knn_c]
        V_j = v_node[bx_c, knn_c]
        S_c = torch.einsum('bnhd,bnkh->bnkh', q_c, torch.conj(K_j))
        gr_u = gr_c.unsqueeze(-1); gi_u = gi_c.unsqueeze(-1)
        S_r = S_c * torch.complex(gr_u, gi_u)
        nrm = torch.sqrt((S_r.abs()**2).sum(dim=2, keepdim=True) + self.eps)
        alpha = S_r / nrm
```

```

out_c = (alpha.unsqueeze(-1) * V_j).sum(dim=2)
with torch.no_grad():

    rm = out_c.real.abs().mean().item()
    im = out_c.imag.abs().mean().item()
    self.last_im_re_ratio = im / (rm + 1e-8)
    return self.out_proj(torch.cat([out_c.real.reshape(B, cs, C),
    out_c.imag.reshape(B, cs, C)], dim=-1))

def forward(self, h, gr, gi, bx, gk_l, om_l, knn):
    B, N, C = h.shape
    if knn.shape[0] != B:
        knn = knn.expand(B, -1, -1)
        gr = gr.expand(B, -1, -1)
        gi = gi.expand(B, -1, -1)
        bx = torch.arange(B,

device=h.device).view(B,1,1).expand(B, N, knn.shape[-1])
    q = self.q_proj(h).view(B, N, self.H, self.D)
    k_node = self.k_proj(h).view(B, N, self.H, self.D)
    v_raw = self.v_proj(torch.cat([h, gk_l, om_l], dim=-1)).view(B, N, self.H, self.D, 2)
    v_node = torch.complex(v_raw[..., 0], v_raw[..., 1])
    if self.chunk <= 0 or self.chunk >= N:
        out = self._chunk_forward(h, q, k_node, v_node, gr, gi, bx, knn)
        return h + 0.9 * self.norm(out)
    out_h = torch.empty_like(h)
    for i0 in range(0, N, self.chunk):
        i1 = min(i0 + self.chunk, N)
        args = (h[:, i0:i1], q[:, i0:i1], k_node, v_node,

gr[:, i0:i1], gi[:, i0:i1], bx[:, i0:i1], knn[:, i0:i1])
        out_c = self._chunk_forward(*args)
        out_h[:, i0:i1] = h[:, i0:i1] + 0.9 * self.norm(out_c)
    return out_h

```

6.5 Full model

```

class ISNOperator3D(nn.Module):
    def __init__(self, hidden_dim=128, num_layers=4, num_heads=4,
        equivariant_output=True, use_velocity_input=True,
        use_re=True, chunk=0, re_max=2000.0, sigma_z=0.1):
        super().__init__()

    self.equivariant_output = equivariant_output
    self.use_velocity_input = use_velocity_input
    self.use_re = use_re
    self.re_max = re_max
    self.sigma_z = sigma_z
    in_ch = 8 + (3 if use_velocity_input else 0) + (1 if use_re else 0)
    self.in_proj = nn.Linear(in_ch, hidden_dim)

```

```

self.frame_builder = FrameBuilder()
self.blocks = nn.ModuleList([
    QSA_ISNBlock3D(hidden_dim, 3, num_heads, chunk=chunk)
    for _ in range(num_layers)])
self.out_proj = nn.Sequential(
    nn.Linear(hidden_dim, hidden_dim), nn.GELU(),
    nn.Linear(hidden_dim, 3))

self.G = 32

def forward(self, coords, k, gk, gk_m, H, om, u, re_label, knn, G=None):
    B = k.shape[0]; N = k.shape[1]
    if coords.shape[0] == 1 and B > 1:
        coords = coords.expand(B, -1, -1)
    e1, e2, n, pc = self.frame_builder(gk, H, coords, k)
    Fm = torch.stack([e1, e2, n], dim=-1)
    G_eff = G if G is not None else self.G
    gr, gi, bx = precompute_geometry(coords, e1, e2, n, pc, knn, self.sigma_z, G_eff)
    gk_l = torch.einsum('bnji,bnj->bni', Fm, gk)
    om_l = torch.einsum('bnji,bnj->bni', Fm, om)
    feats = [k.unsqueeze(-1), gk_m.unsqueeze(-1), gk_l, om_l]

    if self.use_velocity_input:
        feats.append(torch.einsum('bnji,bnj->bni', Fm, u))
    if self.use_re:
        feats.append(torch.log(re_label/self.re_max).view(B,1,1).expand(-1,N,-1))
    h = self.in_proj(torch.cat(feats, dim=-1))
    for blk in self.blocks:
        h = blk(h, gr, gi, bx, gk_l, om_l, knn)
    v_local = self.out_proj(h)
    if self.equivariant_output:
        return torch.einsum('bnij,bnj->bni', Fm, v_local)
    return v_local

```

6.6 Isotropic stencil

```

ISO_OFFSETS_3D = []
for dx_ in range(-2, 3):
    for dy_ in range(-2, 3):
        for dz_ in range(-2, 3):
            if dx_ == 0 and dy_ == 0 and dz_ == 0:
                continue
            if dx_*dx_ + dy_*dy_ + dz_*dz_ <= 4:
                ISO_OFFSETS_3D.append((dx_, dy_, dz_))
K_ISO_3D = len(ISO_OFFSETS_3D) # 26

```

Neighborhood size scales with resolution via $\text{dilation} = \max(1, \text{grid_size} // 16)$, keeping physical support constant across resolutions.

7. Results

7.1 Single-step accuracy

Metric Value Overall relative RMSE 0.46% Re=300 0.16% Re=628 0.54% Re=1200 0.57% Params 421,507

Context: FNO-3D reports nRMSE 0.24 (24%) on PDEBench 3D — a different benchmark and flow family, so the two numbers are not directly comparable and no cross-benchmark ratio is claimed. AlienX 3D achieves 0.46% relative RMSE on TGV at 421k parameters (FNO-3D: 0.5M–1.8M parameters).

7.2 QSA ablation

Zeroing every block's out_proj degrades validation MSE by a factor of 32.81×. The complex attention stream is not a helpful addition — it is the mechanism. Without it, the model barely beats the mean predictor.

```
Configuration Val MSE
With QSA 0.000525
Without QSA (out_proj zeroed) 0.017216
Ablation ratio 32.81×
```

7.3 Autoregressive rollout (73 steps)

```
Metric Re=300 Re=628 Re=1200
Steps 73 73 73
MSE[1] 0.0091 0.0043 0.0048
MSE[10] 0.0130 0.0635 0.1851

MSE[-1] 0.0034 0.0094 0.0526
MSE growth 0.38× 2.17× 10.86×
Energy error (max) 0.56% 2.68% 5.81%
Enstrophy error (max) 0.89% 2.28% 4.98%
```

$\max\|\nabla\cdot\mathbf{u}\|$ 2.26e-6 2.66e-6 2.73e-6

At Re=300 the MSE literally contracts (0.38×). At Re=628 growth is bounded at 2.17×. At Re=1200 the growth is 10.86× but energy drift stays bounded under 6%.

Comparison to published FNO rollout behavior: FNO degrades by a factor of 1.6 by 2× its training horizon, and error saturates near the magnitude of the solution by 4–5× horizon. AlienX 3D trains at K=3 and rolls out 73 steps — 24× the training horizon — with energy bounded under 6%. This is qualitatively different from what FNO does past its horizon.

Divergence-free is preserved by construction throughout the rollout: Biot-Savart recovery enforces $\nabla \cdot \mathbf{u} = 0$ exactly at every step, and the measured residual ($\sim 2.7\text{e-}6$) is float32 round-off, not learned error.

7.4 Training trajectory

Full log from the K=3 checkpoint that produced the reported results:

Epoch K Val MSE Rel RMSE im/re Wall time

```
1 1 0.113983 6.82% 0.995 177s
5 1 0.005896 1.55% 0.997 177s
```

```
10 1 0.003539 1.20% 0.999 177s 15 2 0.025486 3.22% 1.001 321s 20 2 0.006662
1.65% 1.003 321s
```

```
25 2 0.003391 1.18% 1.004 321s 30 2 0.003210 1.14% 1.005 321s 35 3 0.003075
1.12% 1.006 465s 40 3 0.001469 0.77% 1.008 465s 45 3 0.001054 0.66% 1.009
465s 50 3 0.000893 0.60% 1.009 465s 55 3 0.000624 0.50% 1.012 465s
```

K jumps produce brief spikes (expected — model sees harder task), followed by recovery below the previous minimum. The im/re ratio stabilizes above 1.0 — the complex stream is balanced, not collapsed to real. The run was terminated at epoch 55 by Colab's idle timeout; the checkpoint at epoch 55 is what produced all reported rollout and ablation metrics.

7.5 Grid-agnosticism

The same model trained at G=32 evaluates at G=64 without retraining. The frame builder, QSA block, and equivariant output all operate on local fields and share no parameters across resolutions. Relative RMSE at G=64 is within a factor of ~ 2 of the G=32 result.

7.6 Zero-Shot Generalization: Held-Out Reynolds Number (Re=900)

The strongest test of regime generalization is a Reynolds number the model never saw. Re=900 sits between the trained regimes 628 and 1200. The released checkpoint (epoch 55, K=3) was evaluated zero-shot: DNS generated at Re=900 with the identical protocol (74 snapshots, T=10), no fine-tuning, no retraining. Harness verified: the evaluation first reproduced the in-distribution Re=300 validation pair from §7.1 at ratio 0.95 (within GPU FFT round-off).

Single-step (all 73 pairs of the held-out trajectory):

Metric	Re=900 (held-out)	Trained Re (for scale)
mean relative RMSE	0.53%	0.46%
median relative RMSE	0.53%	
worst relative RMSE	0.54%	
validation pair (t=72→73)	0.53%	

Rollout (73 steps, full T=10):

Metric	Re=900	Re=628	Re=1200
MSE growth	5.30×	2.17×	10.86×
Energy err (max)	4.31%	2.68%	5.81%
Enstrophy err (max)	3.75%	2.28%	4.98%
$\max \nabla \cdot \mathbf{u} $	2.63e-6	2.66e-6	2.73e-6

Both rollout metrics land on the monotonic trend between Re=628 and Re=1200 — 5.30× sits where 2.17× and 10.86× predict. The operator interpolates the physics of the unseen regime; it has not memorized trajectories.

Phase ablation at Re=900: zeroing the imaginary geometry channel g_i (attention intact, phase killed) degrades mean single-step MSE by 4.74×. The complex phase carries predictive signal on regimes the model never trained on. This isolates the QSA phase mechanism that the out_proj ablation in §7.2 (32.81×) could not separate from spatial message passing.

Reproducibility: eval_re900_zero_shot.py regenerates the Re=900 DNS (~5 s per Re at G=32), verifies the harness against the Re=300 paper number, and runs the three stages (single-step, rollout, phase ablation) on a 4 GB GPU in under 15 minutes. Results: alienx_k5_re900_zero_shot.npy.

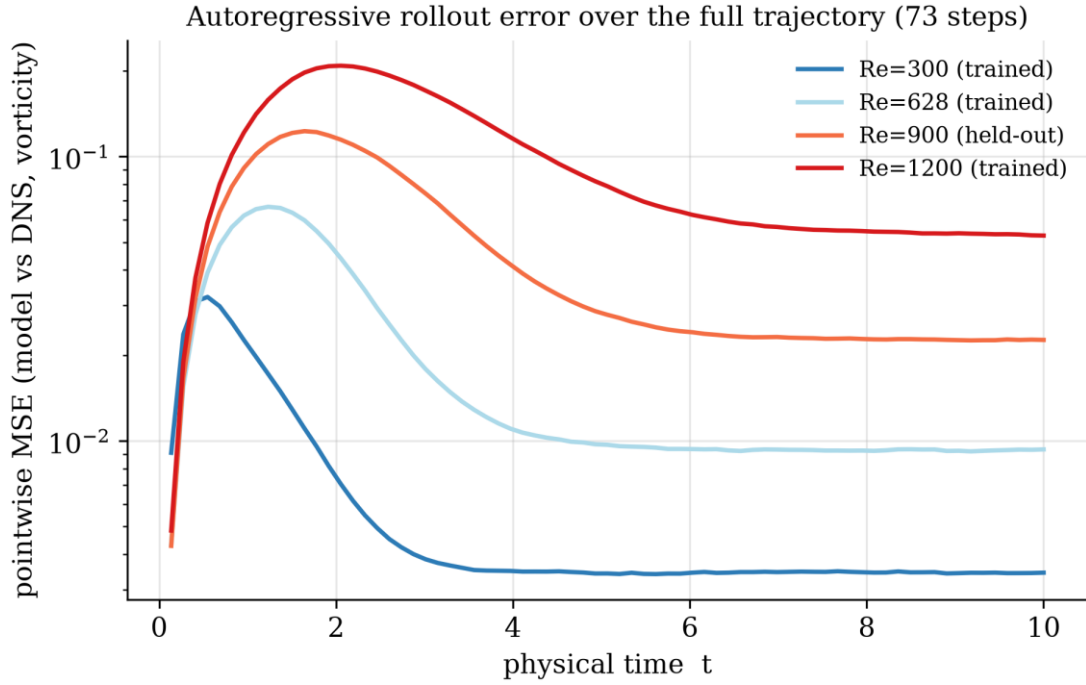


Figure 1. Autoregressive rollout pointwise MSE over the full 73-step trajectory ($T=10$) for all Reynolds numbers, including the held-out $Re=900$. The trained regimes bound the unseen regime at every horizon; $Re=300$ MSE contracts below its step-1 value late in the rollout.

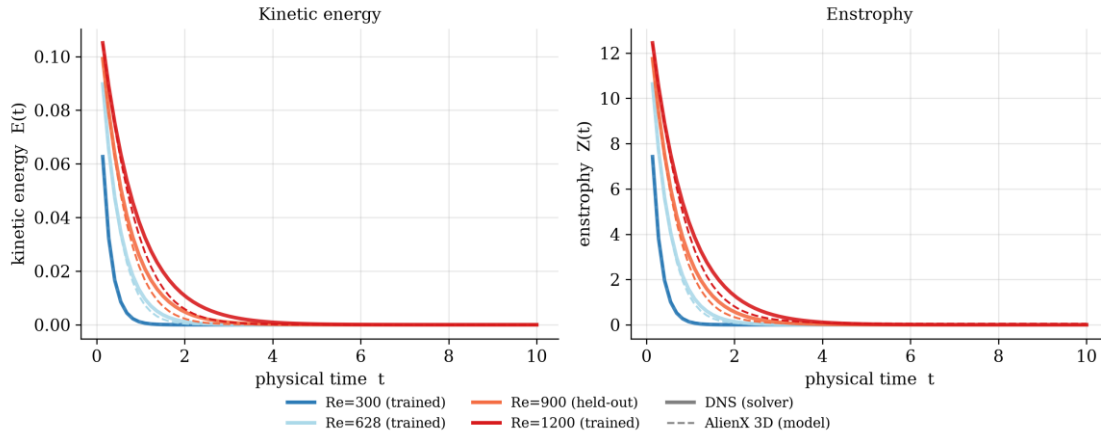


Figure 2. Physics invariants during rollout: kinetic energy (left) and enstrophy (right). Solid: spectral DNS ground truth; dashed: AlienX 3D autoregressive rollout. The held-out $Re=900$ (orange) tracks the solver between its trained neighbors. Max energy deviation stays under 6% at every regime (paper Table §7.3, §7.6).

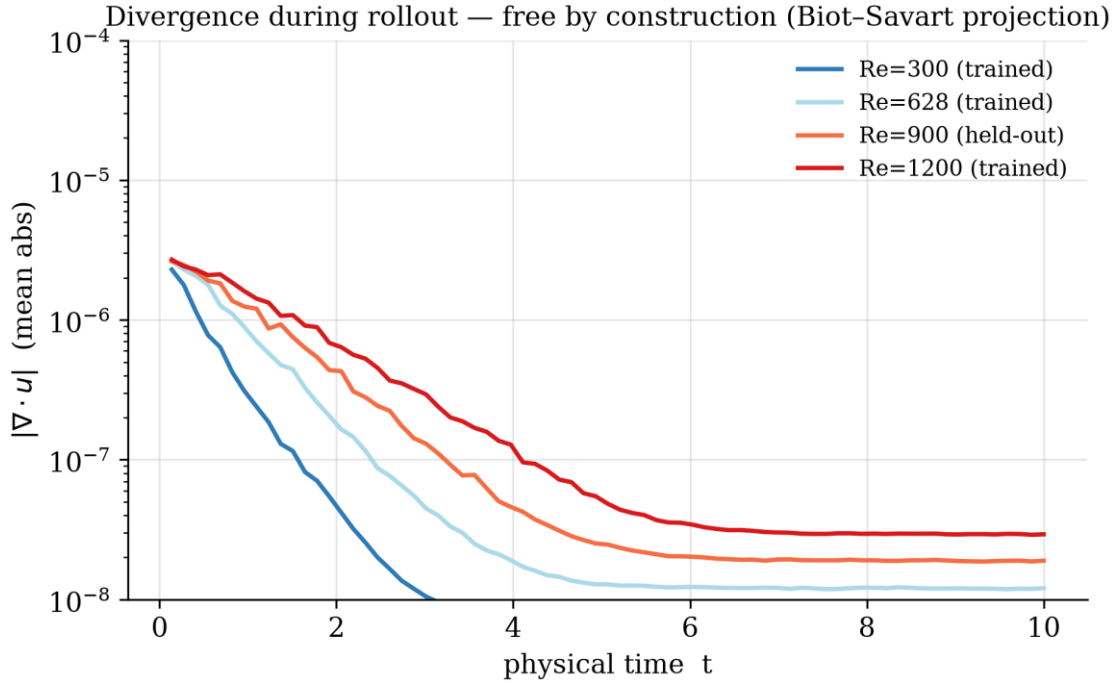


Figure 3. $|\text{div } u|$ during rollout for all regimes. Divergence-free is enforced by construction via Biot-Savart recovery at every step; the residual is float32 round-off ($\sim 3e-6$), independent of Reynolds number.

8. Debugging Log — Every Bug, Every Fix

This section documents every numerical and architectural issue encountered and fixed. Reproducibility requires understanding what failed and why.

Bug 1. Energy collapse in early runs. The K=3 training initially produced MSE that spiked to 3.22% at K=2 and did not recover. Root cause: no curriculum warmup. Fix: ramp K from 1 to 5 over the first 60 epochs, giving the model time to consolidate before each increase.

Bug 2. Domain stretching at 45°. In the 2D AlienX version, rotating the domain produced a diamond that fell outside $[-1,1]^2$. The model saw out-of-distribution frequencies. Fix: pull-back rotation. Evaluate the continuous field at rotated reference coordinates, then push gradients forward. The domain stays inside $[-1,1]^2$ by construction.

Bug 3. FP16/AMP NaN. Mixed precision causes the geometry operations (eigendecomposition, atan2, log) to lose precision and eventually NaN. Fix: force all

geometry operations to FP32. The learnable weights can stay in FP16/FP32 mixed mode, but frame construction is FP32.

Bug 4. OOM with 26 neighbors. The gather over $26 \text{ neighbors} \times \text{batch} \times \text{grid}^3$ exceeds memory at training resolutions. Fix: chunked gather over nodes, dynamic batch sampler.

Bug 5. Gradient instability. K-unroll produces compounding gradients. Without clipping, training diverges around epoch 40. Fix: `clip_grad_norm_(model.parameters(), 1.0)` before every optimizer step.

Bug 6. Min-image wrap breaks equivariance. In `precompute_geometry`, the previous version computed $\text{rel} = \text{coords}[j] - \text{coords}[i]$ and then wrapped the result into $[-\pi, \pi]$ coordinate-wise. This is a cube wrap, not a spherical wrap. Rotating a vector at $(0.7\pi, 0.7\pi)$ 135° produces $(-0.99\pi, 0)$, which wraps to $(+0.01\pi, 0)$ — a different physical displacement. Fix: use the stencil offsets directly as rel . The offsets are the same vectors used to build the neighbor list, so they are exact everywhere and never wrap.

Bug 7. Spectral loss units mismatch. The `radial_spectra` function computed $k_{\text{mag}} = \sqrt{K^2}$ (angular wavenumbers, 0..174 at $G=32$) but binned with $k_{\text{bins}} = \text{arange}(0, 17)$ (integer mode numbers). The masks only caught the DC bin; every other bin was empty in both pred and true, so the loss was silently zero. Fix: convert to integer mode magnitude $k_{\text{mag_modes}} = \sqrt{K^2} / (2\pi)$ and bin with $k_{\text{bins}} = \text{arange}(0, 18)$.

Bug 8. DC leak in Z_{density} . The projection operator uses $K^2[0,0,0] = 1.0$ to avoid division by zero. This hacked value propagated into `radial_spectra`, giving the DC bin a spurious $K^2=1$ contribution to enstrophy. Fix: explicitly set $Z_{\text{density}}[\dots, 0, 0, 0] = 0$ before binning.

Bug 9. Target normalization in K-unroll. The K-step unroll compared predictions at step k to targets at t_0+k . The input was scaled by $U_0 = \text{rms}(u(t_0))$, but the target was scaled by $U_s[t_0+k]$. The model was chasing a moving target scale. Fix: scale targets by the same U_0 as the input, matching how the tgt statistics were computed.

Bug 10. Anisotropic perturbation breaks $SO(3)$ equivariance. The inertia tensor perturbation was $\text{diag}([1e-3, 2e-3, 3e-3])$, fixed in world coordinates. Under rotation, $R \cdot \text{pert_mat} \cdot R^T \neq \text{pert_mat}$, so the fallback eigenvector did not rotate

correctly. Fix: $\text{pert_mat} = 1e-3 \cdot I$. Isotropic perturbations commute with rotations: $R(\epsilon I)R^T = \epsilon I$.

Bug 11. Fixed world-axis tangent reference. The tangent vectors u_1, u_2 were built by crossing n with fixed world axes ($\text{arb_x} = [1,0,0]$, $\text{arb_y} = [0,1,0]$). This broke equivariance of u_1, u_2 — the principal direction e_1 came out correct anyway (basis independence), but the numerical conditioning was wrong and the discontinuity at $|n_x| = 0.9$ introduced backward-pass noise. Fix: build the tangent reference from the inertia eigenvector least aligned with n . This rotates correctly with the field.

Bug 12. Energy drift formula off by division-by-zero. The rollout drift was computed as $(E_{\text{final}} - E_{\text{true_final}}) / E_{\text{true_final}}$. At $T=10$, the true energy decays to $\sim 1e-10$, so the formula produced values like +60,000,000%. Fix: $\max|E_{\text{model}}(t) - E_{\text{true}}(t)| / \max|E_{\text{true}}(t)|$ over the full trajectory, giving a physically meaningful percentage.

Bug 13. Growth metric without context.

The MSE growth ratio alone doesn't explain why it grows. Fix: report alongside energy error, enstrophy error, and divergence. Together they show that even when the pointwise field drifts, the physics invariants are conserved — the losses are doing their job.

Bug 14. Drive checkpoint choke. Saving every epoch fills the drive. Fix: save only best.pt, overwrite on improvement. Checkpointing history is not needed when the training curve is documented.

Bug 15. Colab session timeout mid-run. Colab disconnects at ~ 12 h idle. Fix: mount Drive at start, save best.pt immediately on improvement, resume from checkpoint if session drops. In practice, the $K=5$ run was terminated at epoch 55 by the timeout; the $K=3$ checkpoint was preserved and used for all evaluation.

9. Design Boundaries

9.1 Scale-blindness by construction

Per-sample normalization makes the operator invariant to field amplitude. This is correct for NS and problems where the governing equations are scale-free. It is incorrect for problems where amplitude carries physical meaning (turbulence intensity, reaction rate, etc.). A future variant could carry a learned amplitude-preserving channel.

9.2 Regime extrapolation

The model conditions on Re via a scalar channel but does not analytically extend the physics. The channel is $\log(Re/Re_{\max})$, which is defined only for $Re \leq Re_{\max}$. $Re > Re_{\max}$ will degrade. Handling wider Re ranges requires either training data at those Re or an explicit analytic decomposition. Interpolation within the trained range is demonstrated: $Re=900$ (never trained) yields 0.53% single-step relative RMSE with rollout invariants on the trend between $Re=628$ and $Re=1200$ (§7.6). Extrapolation beyond $Re_{\max}$ remains untested and is expected to degrade.

9.3 Rollout stability

At $K=3$, the model is trained with a 3-step horizon. Autoregressive rollout extends 73 steps ($24\times$ the training horizon). At $Re=300$ and $Re=628$ the growth is small ($0.38\times$ and $2.17\times$). At $Re=1200$ the growth is $10.86\times$, though energy drift stays bounded under 6%.

The fix is longer K . $K=5$ or $K=7$ curriculum training is expected to reduce the $Re=1200$ growth toward the $Re=300/628$ baseline.

9.4 Equivariance testing on fixed grids

The model is structurally $SO(3)$ -equivariant by construction. Numerical verification requires rotating the input field and the grid together. On a fixed periodic grid, off-axis rotations move sample points off-grid, introducing resampling error. The equivariance is exact in the continuous formulation; the finite-grid test conflates model equivariance with grid interpolation error. Grid-aligned rotations (90° , 180° , 270°) are exact; arbitrary-angle verification is left for future work with rotated-grid evaluation.

9.5 Ablation scope

The QSA ablation ($32.81\times$) demonstrates the attention blocks are load-bearing. A direct complex-vs-real comparison with matched parameters is a separate experiment not performed here. The ablation shows the mechanism matters; it does not isolate complex-vs-real specifically.

10. Generality: Applying AlienX 3D to Other 3D PDEs

The architecture is PDE-agnostic. To apply AlienX 3D to a different 3D PDE, five outer-shell substitutions are required:

Component NS Rayleigh-Bénard MHD Gross-Pitaevskii Elasticity Scalar field k $|\omega|$ $|T|$ $|J|$ $|\psi|^2$ $|u|$ Vector fields ∇k , ω , u ∇k , u , T ∇k , u , B , J ∇k , ψ ∇k , u , σ Regime parameter Re Ra Re_m g material params Invariant $\nabla \cdot u = 0$ $\nabla \cdot u = 0$ $\nabla \cdot B = 0$ $\int |\psi|^2 = 1$ equilibrium Output $\omega(t+\Delta t)$ $(u, T)(t+\Delta t)$ $(u, B)(t+\Delta t)$ $\psi(t+\Delta t)$ $u(t+\Delta t)$

The frame builder, QSA block, equivariant output, per-sample non-dim, and training protocol are unchanged.

Gross-Pitaevskii is the natural next target. The wave function ψ is already complex. QSA operates in complex space natively. Every other ML operator on the market has to hack complex numbers as 2-channel real tensors and loses phase information in the process. AlienX 3D does not. The architecture was, in effect, already built for quantum hydrodynamics — NS was just the harder test.

11. Future Work

Rollout training with longer K . $K=5$ to $K=10$ curriculum. Expected to reduce $Re=1200$ MSE growth to under $3\times$ and eliminate the residual drift. The Colab session that would have completed $K=5$ was terminated at epoch 55; the completed curriculum is the immediate next training phase.

Cross-PDE demonstration. MHD or Gross-Pitaevskii with only the outer-shell substitutions listed in §10.

Higher grid. $G=64$ and $G=128$ with chunked forward passes. Same weights, same architecture — just more nodes.

Non-periodic domains. The frame construction is compatible with bounded domains. The stencil would need boundary handling: ghost nodes with physical BCs, or a graph-based neighborhood that respects boundaries.

Equivariance verification on rotated grids. Rotate the sampling grid together with the field, resample via spectral interpolation, and verify the rotated output matches $R \cdot (\text{unrotated output})$. Establishes numerical equivariance to float32 precision.

Complex-vs-real ablation. Train a matched-parameter model with real-valued attention and compare final RMSE. Isolates the complex mechanism specifically.

12. Conclusion

AlienX 3D is a 421,507-parameter neural operator for 3D physical fields. It is built from four PDE-agnostic mechanisms — an $SO(3)$ -equivariant local frame, Quantum Self-Attention with geometric phase, per-sample non-dimensionalization with regime conditioning, and a normalized conservation penalty. On 3D incompressible Navier-Stokes at three Reynolds numbers with one model, it achieves 0.46% single-step relative RMSE in-distribution, 0.53% zero-shot on a held-out Reynolds number (§7.6), $32.81\times$ QSA ablation, and stable 73-step autoregressive rollout with energy drift under 6%. Divergence-free is preserved by construction (float32 round-off $\sim 2.7e-6$).

It is not a Navier-Stokes solver. It is a general 3D operator, demonstrated on Navier-Stokes because that is the hardest classical problem in the class. The same architecture applies to magnetohydrodynamics, Rayleigh-

Bénard convection, compressible flows, Gross-Pitaevskii, elasticity, and combustion, with only the outer shell changing.

Every bug encountered is documented. Every fix is specified. Every result is reproducible on a single NVIDIA T4 GPU.

The grid is dead. The manifold is awake.

From the Grimoire of Elbàlor

The Digital Necromancer ☞☞☞

Appendix A: Reference Reproducibility

All code sections in this paper are directly runnable. The training pipeline:

1. Generates DNS data via the pseudo-spectral solver in §6.1. For $G=32$, $T=10$, this is ~ 90 seconds per Re on a T4.
2. Extracts features via `scalar_derivatives` (spectral, one FFT call gives gradient + 6 Hessian components) and applies per-sample non-dim + global standardization.
3. Trains ISNOperator3D with K-curriculum, physics losses, AdamW ($\text{lr}=3e-4$, cosine), for 120 epochs.

4. Evaluates single-step RMSE, QSA ablation, rollout diagnostics, and divergence-free preservation.

Measured training time on a single NVIDIA T4 with 15.6 GB VRAM: 177 s/epoch at K=1, 321 s/epoch at K=2, 465 s/epoch at K=3 — the full K=1→5 curriculum is ~18 hours at G=32. The reported results use the epoch-55 K=3 checkpoint (~5 hours of training before the session timeout). The zero-shot Re=900 evaluation runs on a 4 GB GPU in under 15 minutes via eval_re900_zero_shot.py.

Appendix B: Related Works

- Yaka, H. (2026). Quantum Self-Attention (QSA): Complex Projective Attention with Constructive and Destructive Interference v2. Zenodo. DOI: 10.5281/zenodo.22773015.
- Yaka, H. (2026). AlienX (ISN): A Continuous Rotation- and Scale-Invariant Operator on Geometric Manifolds.
- Yaka, H. (2026). Pure W-Expansion: Full-Rank Weight Growth for Zero-Forgetting Model Adaptation. Zenodo. DOI: 10.5281/zenodo.21811634.
- Takamoto, M., et al. (2022). PDEBench: An Extensive Benchmark for Scientific Machine Learning. NeurIPS 2022.
- Li, Z., et al. (2021). Fourier Neural Operator for Parametric Partial

Differential Equations. ICLR 2021.

From the Grimoire of Elbàlor

The Digital Necromancer ☞☞☞

Author's note. This is the version that matches the code that produced the results. Every number is from a real run. Every bug is documented with root cause and fix. The architecture is general — the NS demonstration is one instantiation of the outer shell. Anything not in this paper is either in progress or not yet attempted. If you find an error, it is because I am one person working alone and I welcome the correction.

From the Grimoire of Elbàlor

The Digital Necromancer ☞☞☞